



Software Engineering

Tahir Farooq
tahir.farooq@nu.edu.pk



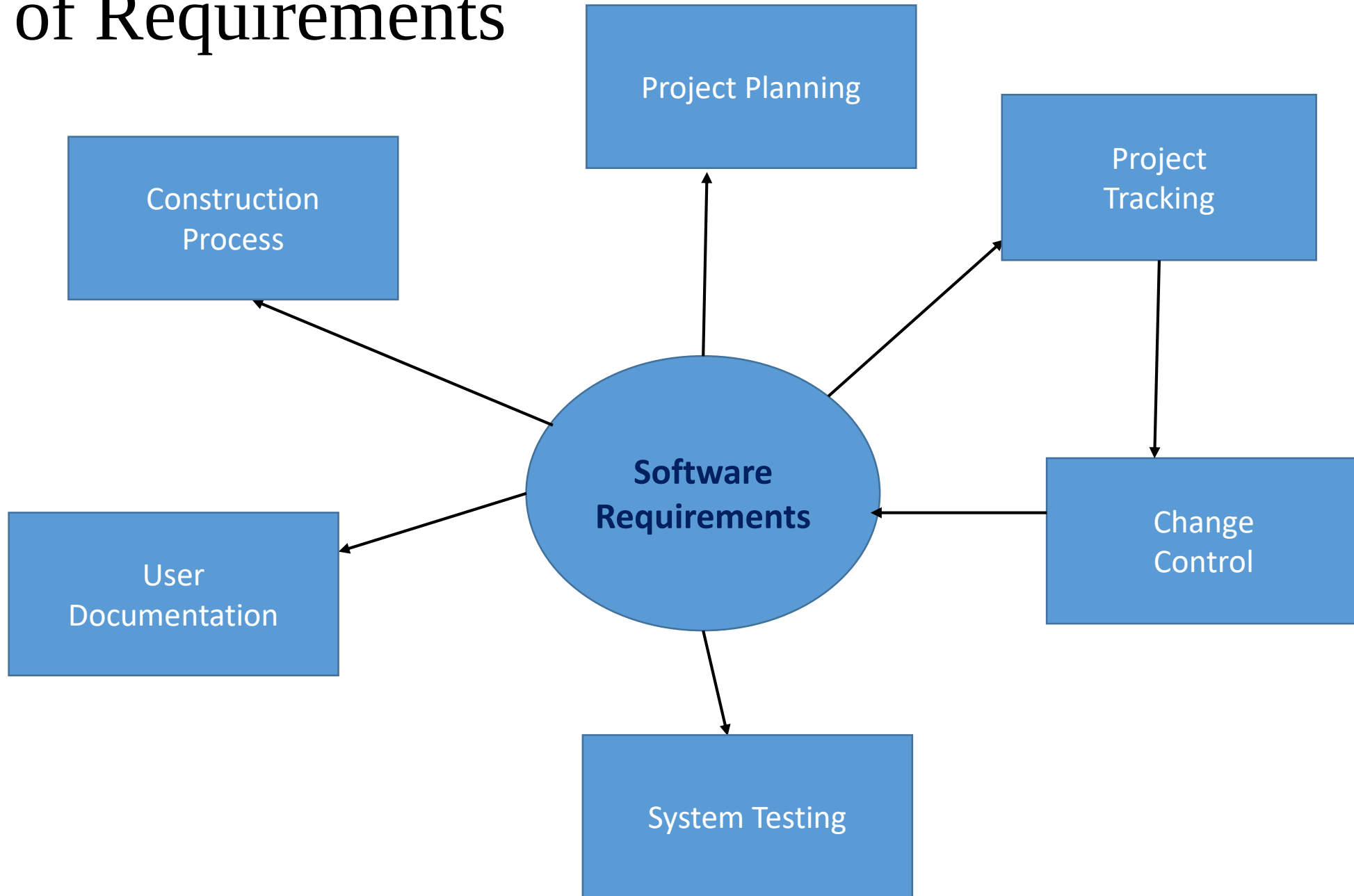
[illegible]

Requirement Engineering

- Requirements are ... A **specification** of what should be **implemented**. They are descriptions of **how** the system should **behave**, or of a system property or attribute. They may be a **constraint** on the development process of the system.

[Sommerville 1997]

Role of Requirements



User requirements and System requirements

User requirements are statements, in a natural language plus diagrams, of **what services** the system is expected to provide to system users and the **constraints** under which it must operate. The user requirements may vary from broad statements of the system features required to detailed, precise descriptions of the system functionality.

System requirements are **more detailed** descriptions of the software system's **functions**, **services**, and operational **constraints**. The system requirements document (sometimes called a functional specification) should define exactly what is to be implemented. **It may be part of the contract between the system buyer and the software developers.**

Mental health care patient information system (Mentcare)

User requirements definition

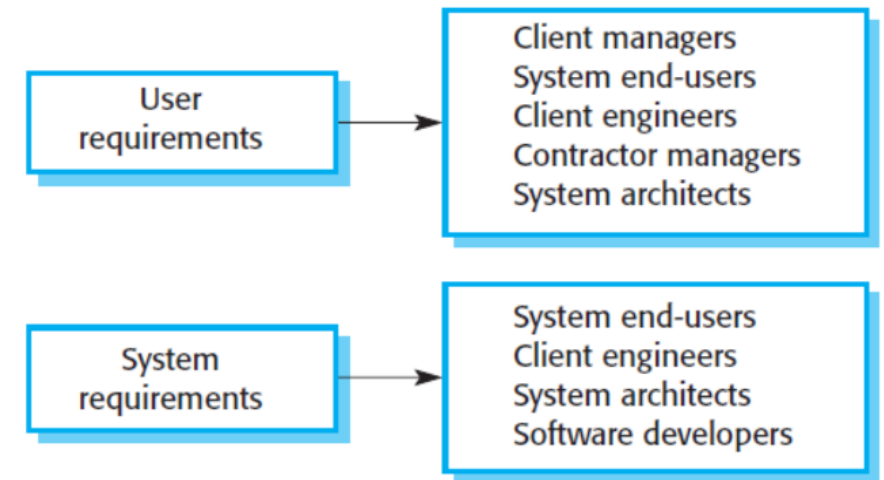
- 1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System requirements specification

- 1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
- 1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
- 1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
- 1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc.) separate reports shall be created for each dose unit.
- 1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

- You need to write requirements at different levels of detail because different types of readers use them in different ways.
- The **readers of the user requirements** are not usually concerned with how the system will be implemented and may be managers who are not interested in the detailed facilities of the system.
- The **readers of the system requirements** need to know **more precisely** what the system will do because they are **concerned** with **how it will support the business processes** or because they are **involved** in the system **implementation**.

In figure there are examples of system stakeholders. Many stakeholders like end-user, manager,



System stakeholders for the Mentcare system include:

Mental health care patient information system (Mentcare)

1. **Patients** whose information is recorded in the system and relatives of these patients.
2. **Doctors** who are responsible for assessing and treating patients.
3. **Nurses** who coordinate the consultations with doctors and administer some treatments.
4. **Medical receptionists** who manage patients' appointments.
5. **IT staff** who are responsible for installing and maintaining the system.
6. A **medical ethics manager** who must ensure that the system meets current ethical guidelines for patient care.
7. **Health care managers** who obtain management information from the system.
8. **Medical records staff** who are responsible for ensuring that system information can be maintained and preserved, and that record keeping procedures have been properly implemented.

Levels of Requirements

- Business Requirements
- User Requirements
- Functional Requirements
- Non-Functional Requirements

Business Requirements

- These are used to state the **high-level** business objective of the organization or customer requesting the system or product
- They are used to **document** main system features and functionalities without going into their nitty-gritty details
- They are captured in a document describing the project **vision** and **scope**

User Requirements

- User requirements add further detail to the business requirements
- User requirements are statements of what **services** the system is expected to provide to system users and the **constraints** under which it must operate.

Functional Requirements

- These are statements of **services** the system should provide, **how the system should react to particular inputs**, and **how the system should behave in particular situations**
 - Depend on the type of software being developed
 - When expressed as user requirements, functional requirements should be written in natural language so that system users and managers can understand them.
 - Functional system requirements expand the user requirements and are written for system developers. They should describe the system functions, their inputs and outputs, and exceptions in detail.
- In some cases, the functional requirements may also explicitly state what the system should not do

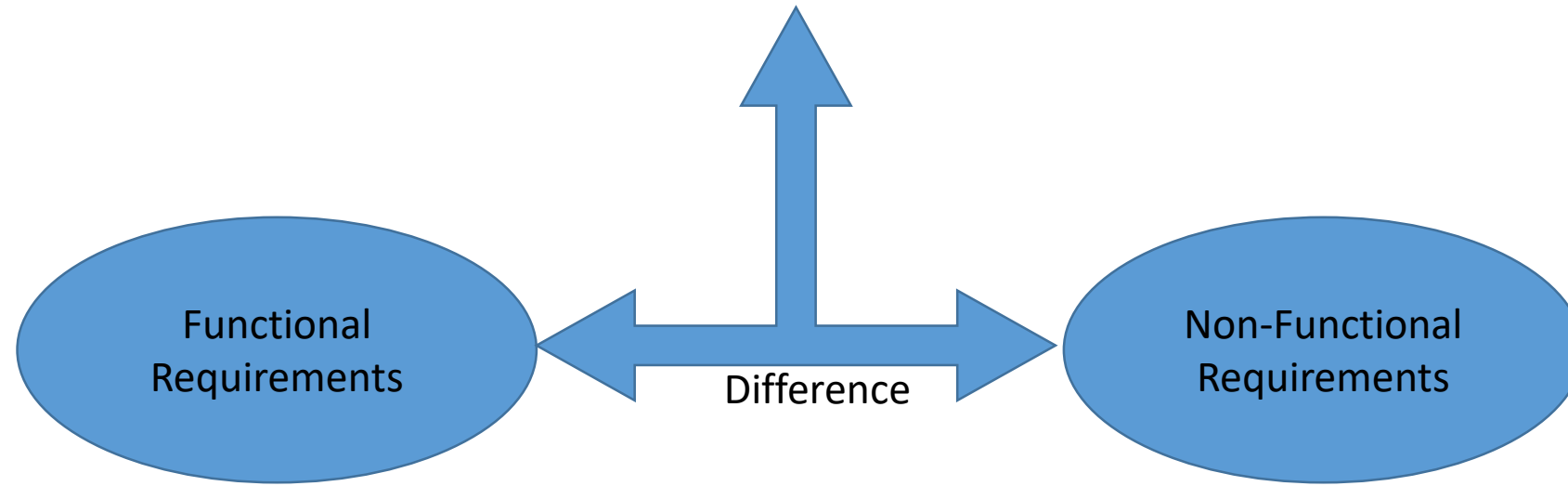
Mentcare (Example)

1. A user shall be able to search the appointments lists for all clinics.
2. The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
3. Each staff member using the system shall be uniquely identified by his or her eight-digit employee number.

Non-Functional Requirements

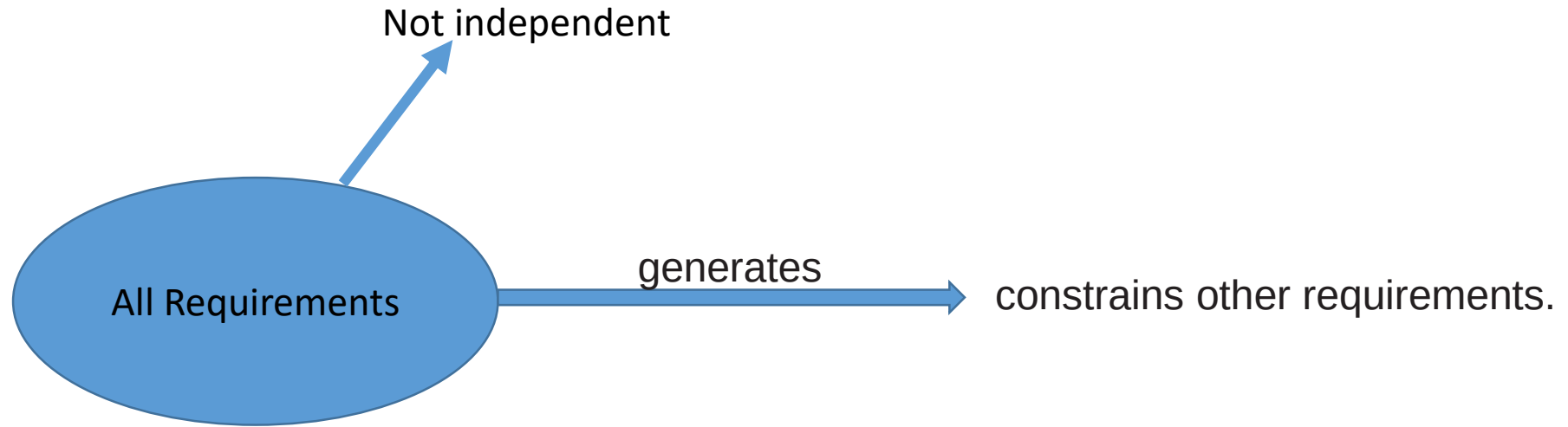
- These are **constraints** on the services or functions offered by the system
- They include **timing constraints**, constraints on the **development process**, and constraints imposed by **standards**
- Non-functional requirements often apply to the system as a whole, rather than individual system features or services
- For example, to ensure that **performance requirements** are met in an **embedded system**, you may have to organize the system to **minimize communications** between components.

In reality difference is not clear

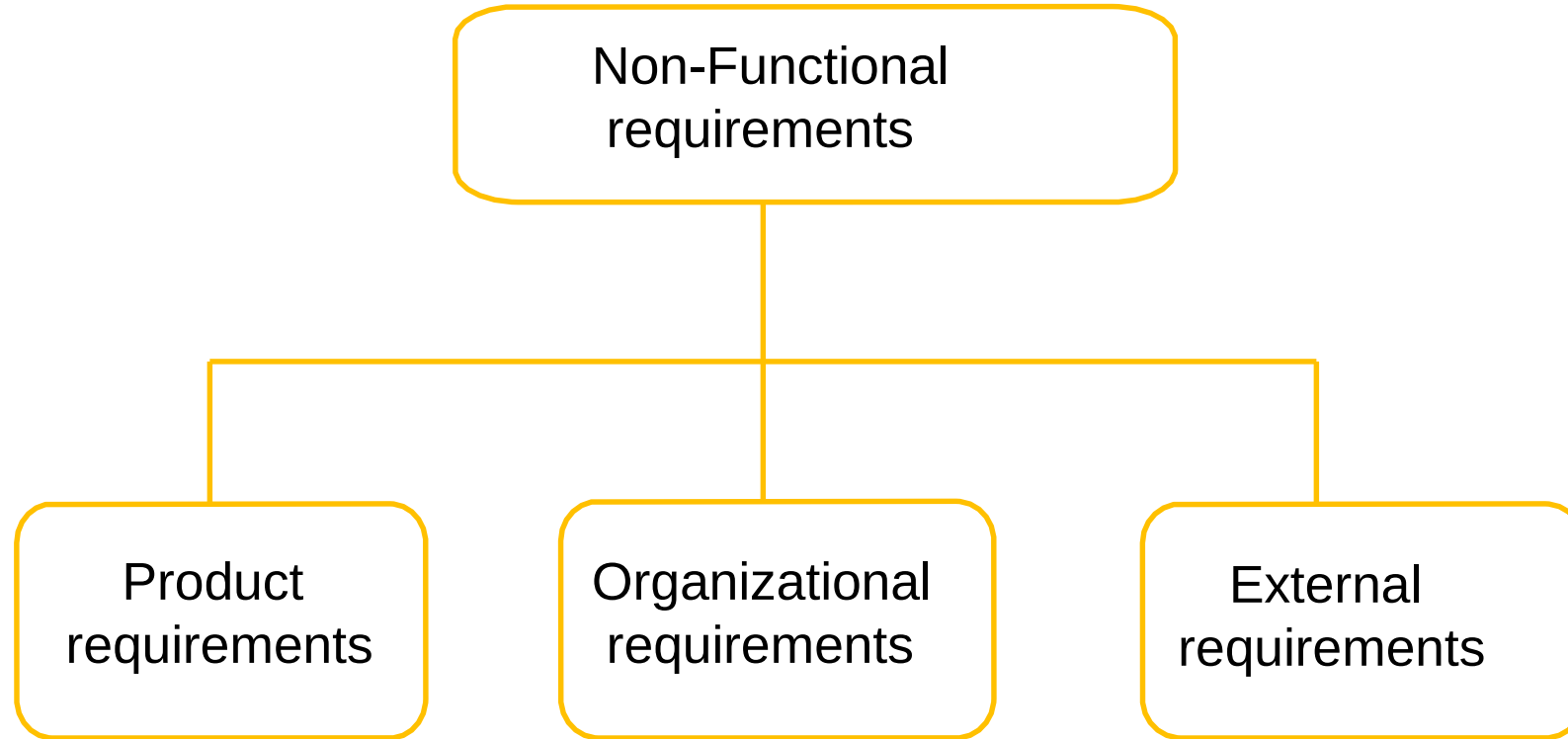


A user requirement concerned with **security**, such as a statement **limiting access to authorized users**, may appear to be a nonfunctional requirement.

However, when developed in more detail, this requirement may generate other requirements that are clearly functional, such as the need to include **user authentication facilities** in the system.



Non-Functional Requirements



Product Requirements

- After completion how much time require to understand full requirements
- User supportive.

- Efficient in space and execution

- Available 24 hours.
- No crash single time

Usability
requirements

Efficiency
requirements

Reliability
requirements

Portability
requirements

Performance
requirements

Space
requirements

Run on multiple
platform(macos, window,
...)

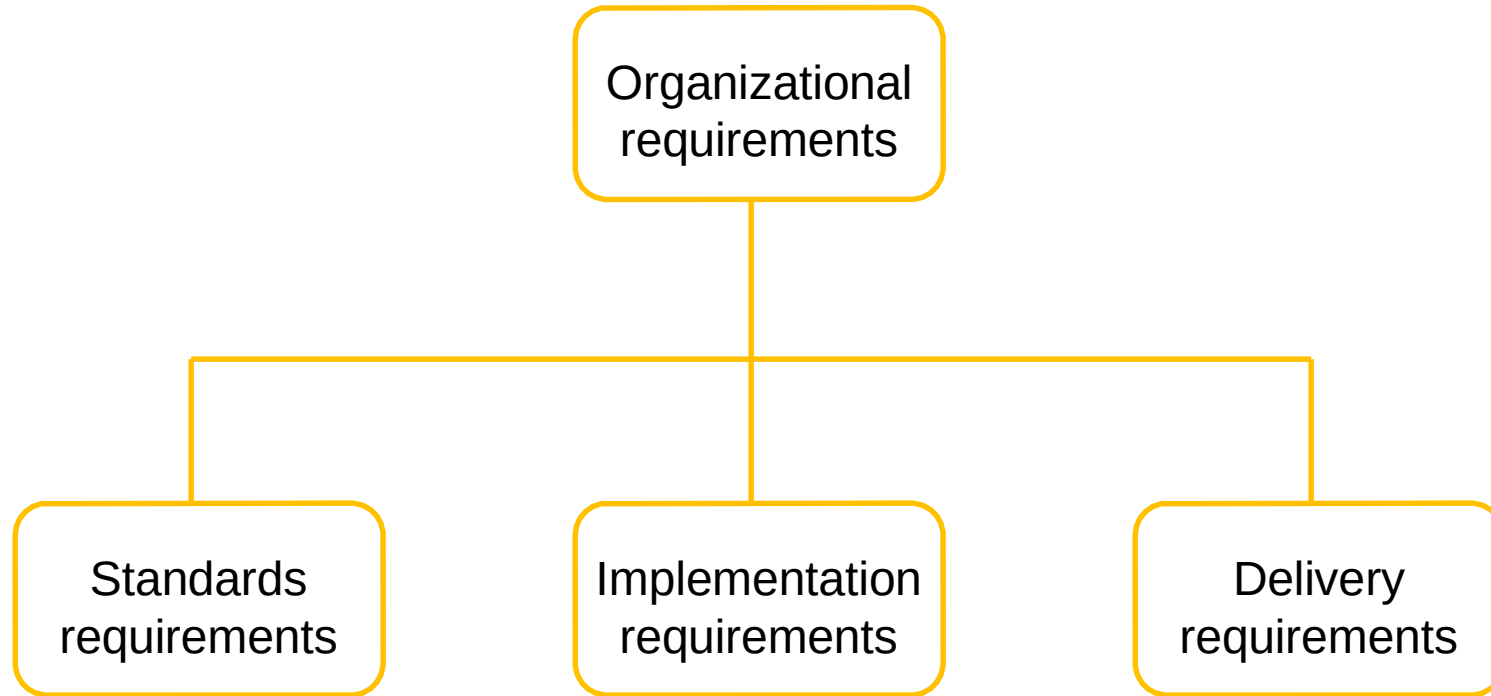
Product Requirements

Reliability and performance

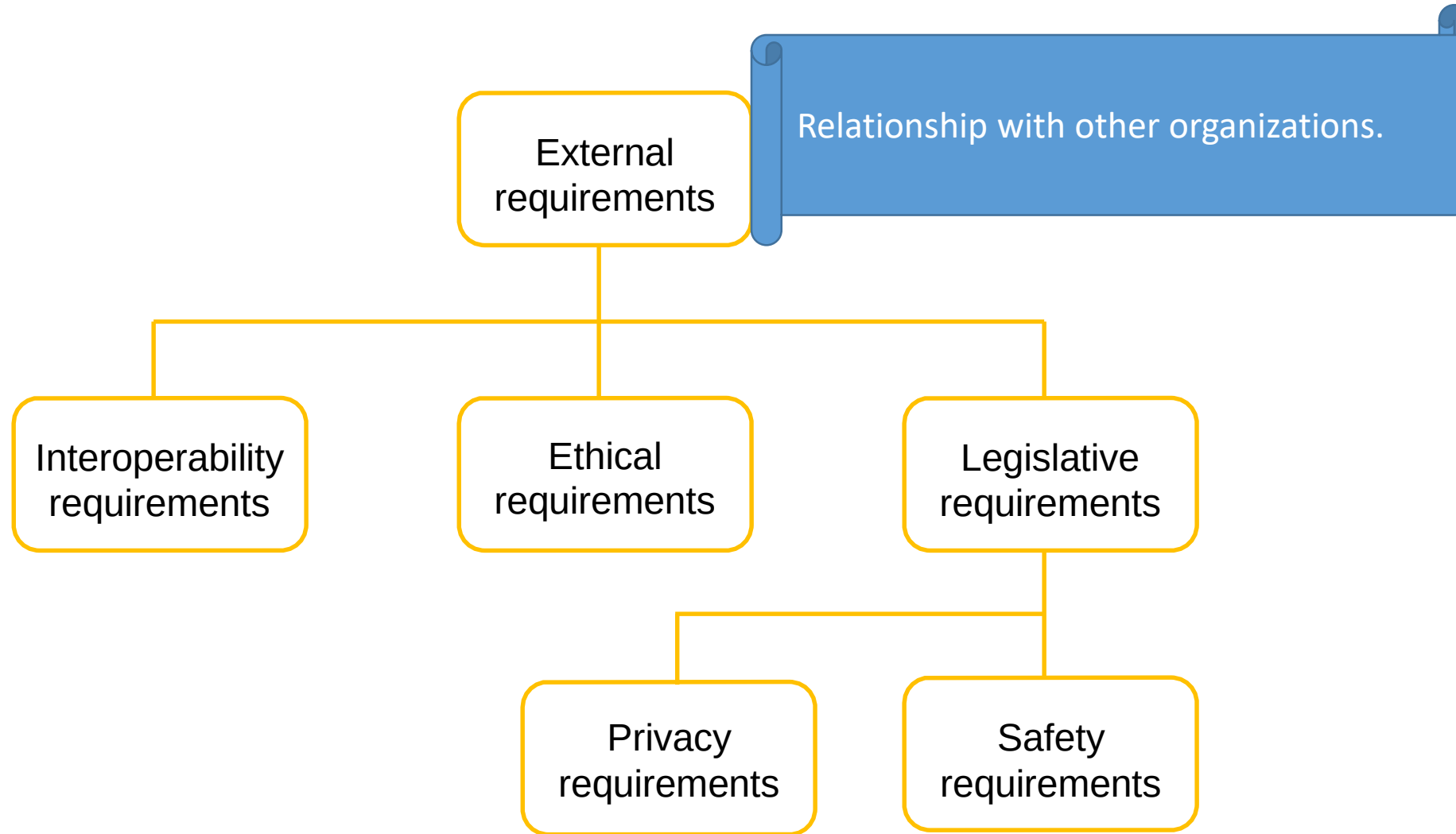
- The system shall allow one hundred thousand hits per minute on the website
- The system shall not have down time of more than one second for continuous execution of one thousand hours

Mean time to failure

Organizational Requirements



External Requirements



External Requirements

Multiple organization are going to use your same product.
Therefore software companies does not give code and documentation

Interoperability
requirements

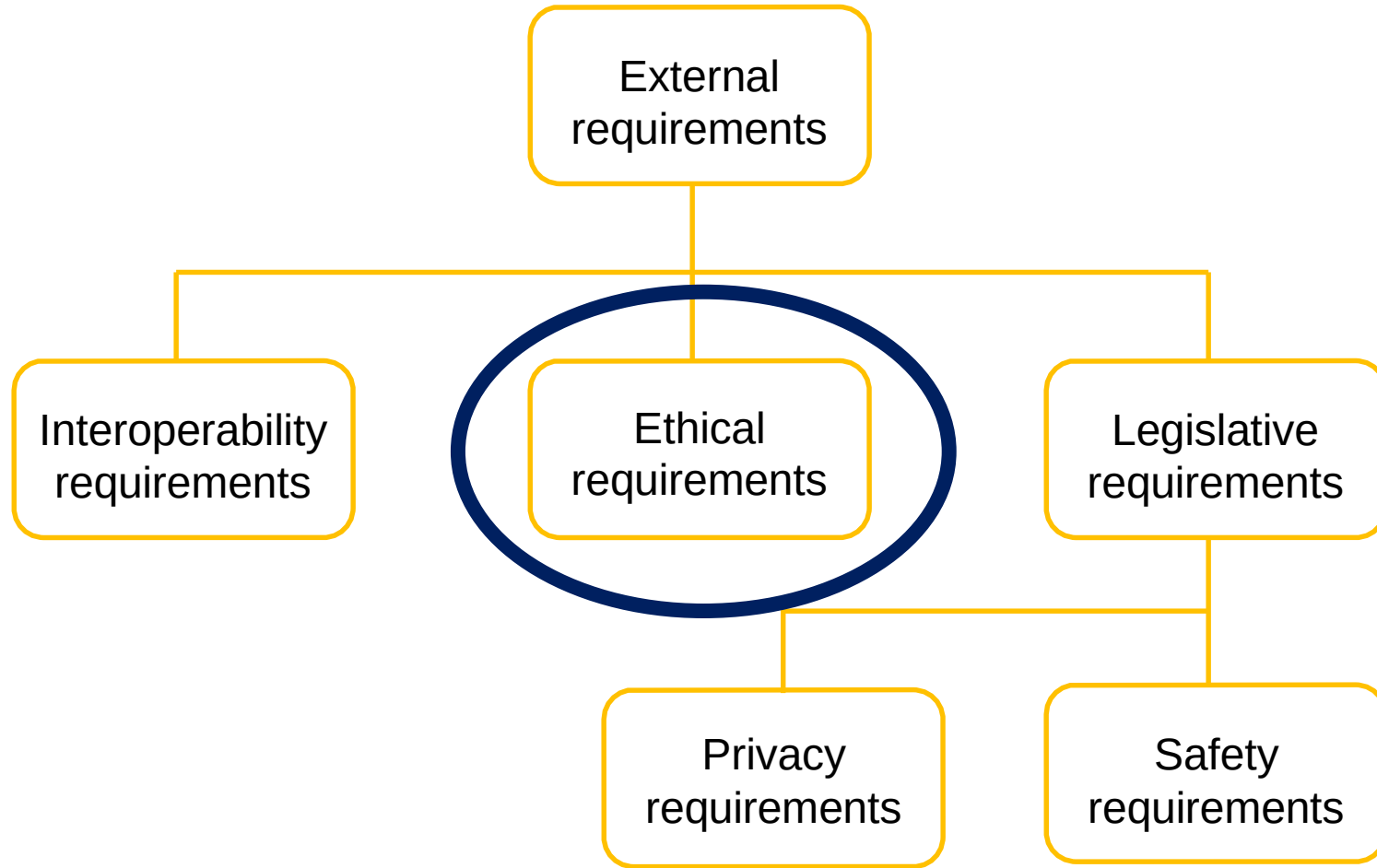
Ethical
requirements

Legislative
requirements

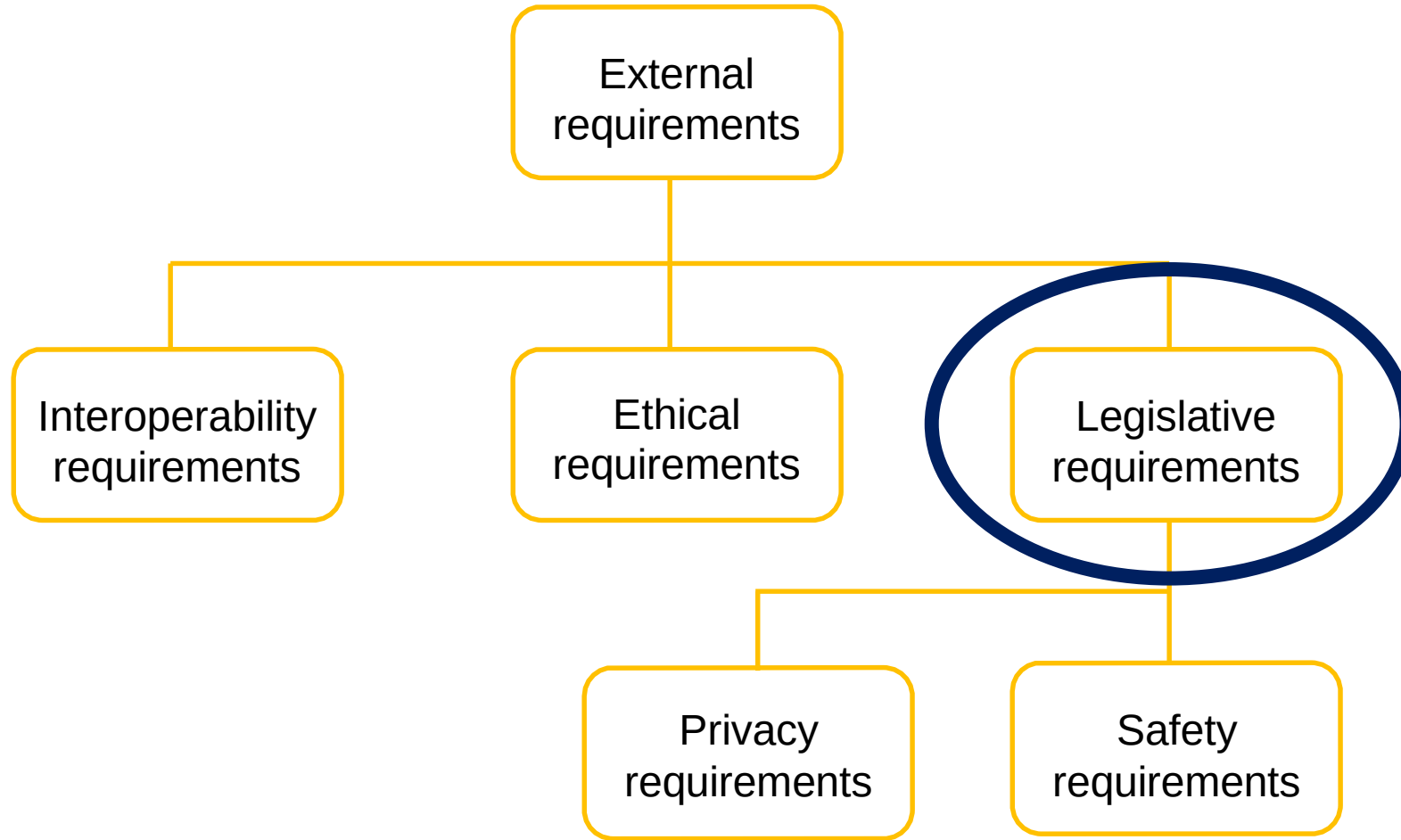
Privacy
requirements

Safety
requirements

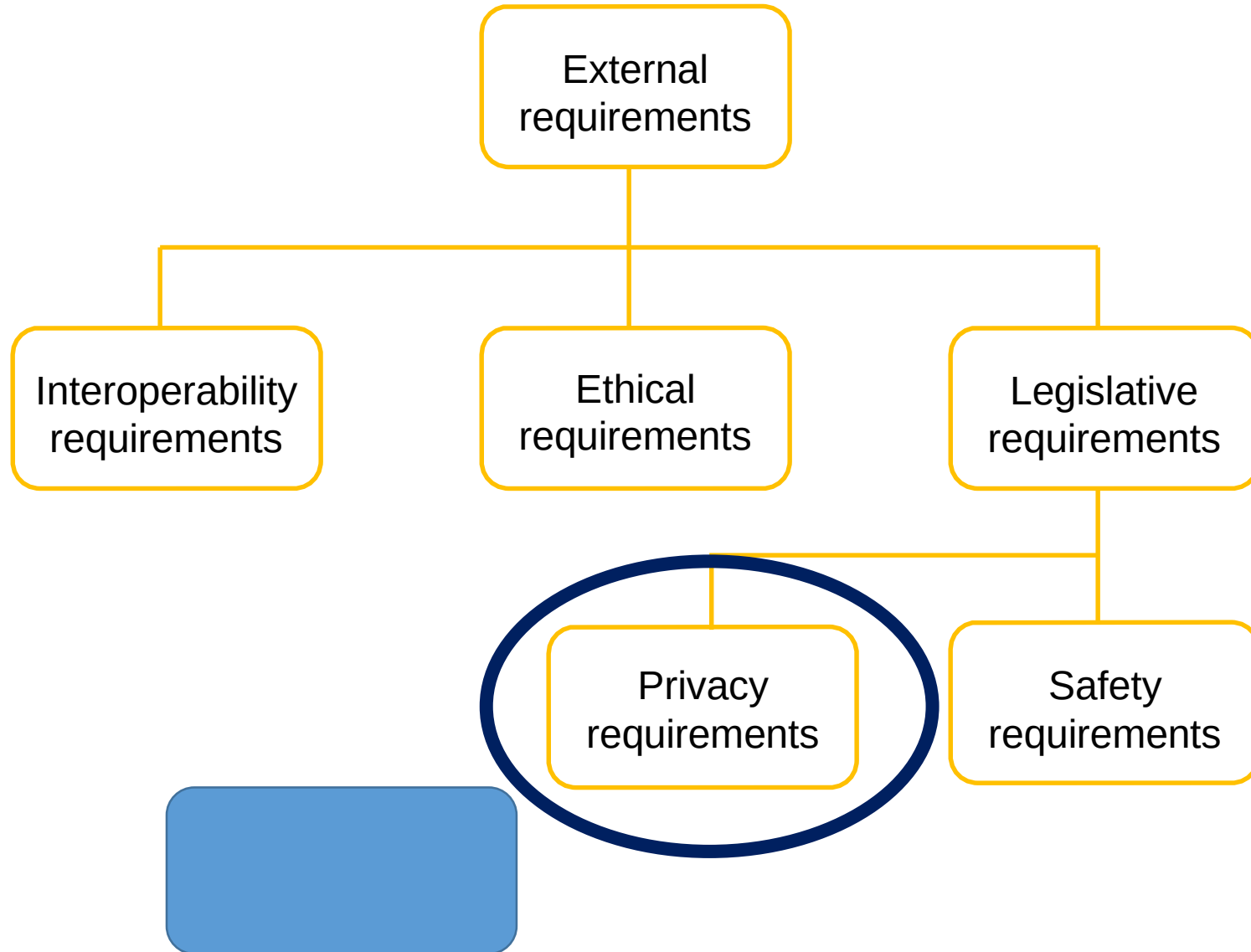
External Requirements



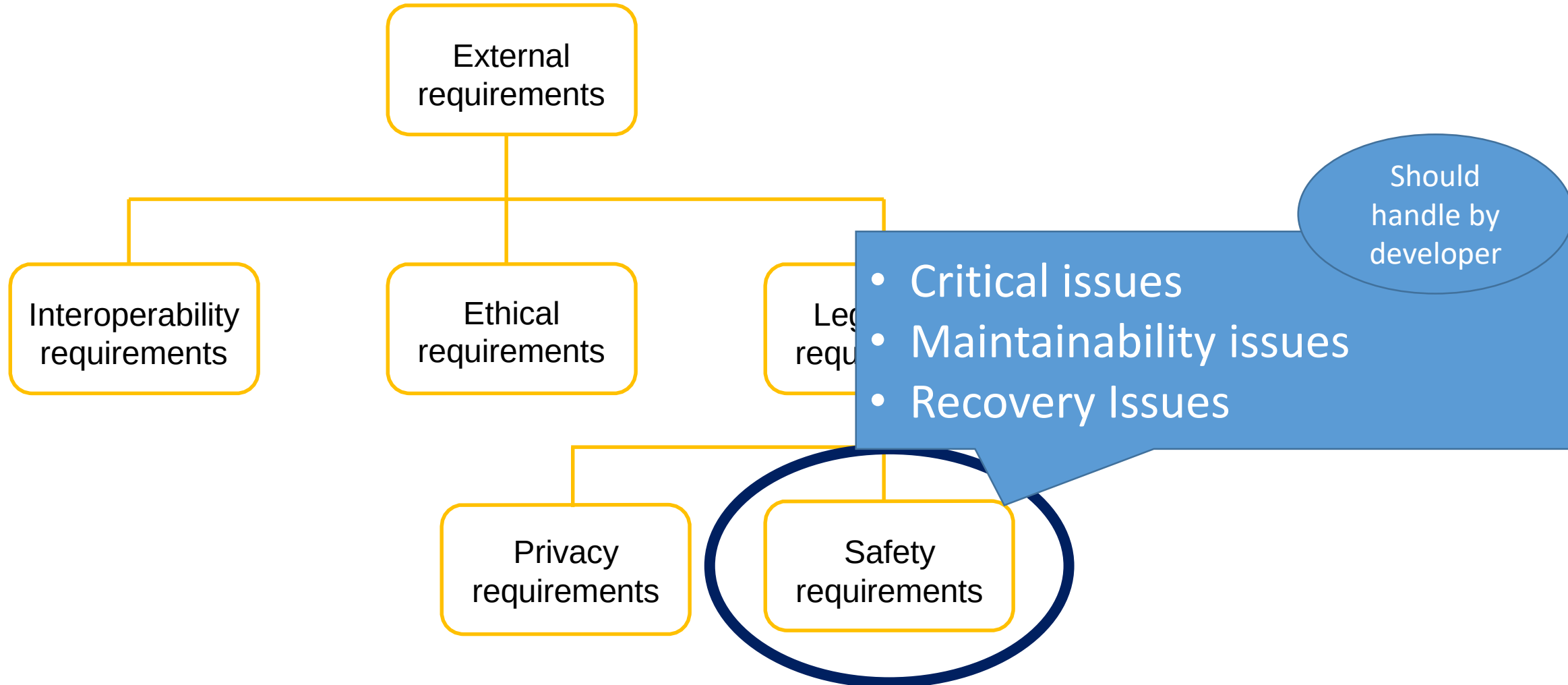
External Requirements



External Requirements

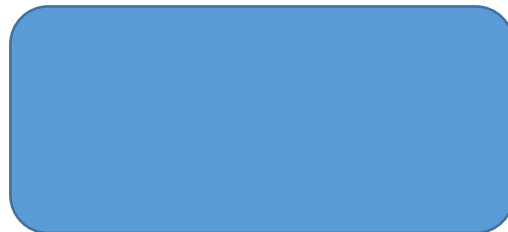


External Requirements



External Requirements

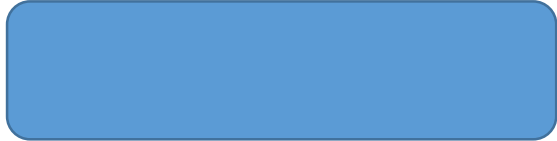
- The system **shall not** disclose any personal **information** about members of the library system to other members except system administrators
- The system **shall** comply with the local and national laws regarding the use of software tools



Domain Requirements

- Requirements that come from the application domain and reflect fundamental characteristics of that application domain
- Can be functional or non-functional
- These requirements, sometimes, are not explicitly mentioned, as domain experts find it difficult to convey them. However, their absence can cause significant dissatisfaction

Domain Requirements

- Domain requirements can impose **strict constraints** on **solutions** 
- This is particularly true for **scientific** and **engineering** domains
- **Domain-specific** terminology can also cause **confusion**

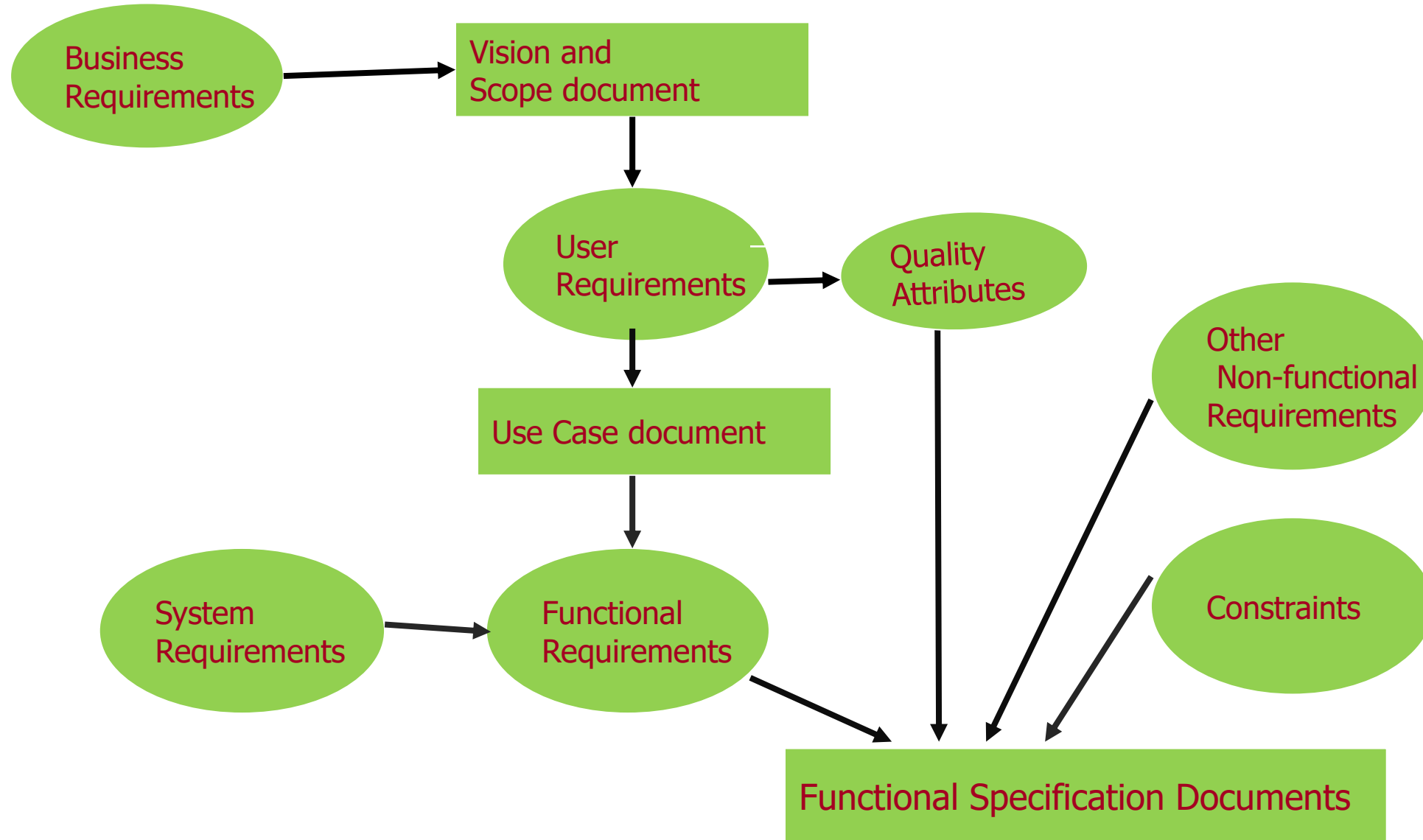
Inverse Requirements

- They explain what the system **shall not do**.
- Many people find it convenient to describe their needs in this manner
- These requirements indicate the **indecisive** nature of customers about certain aspects of a new software product

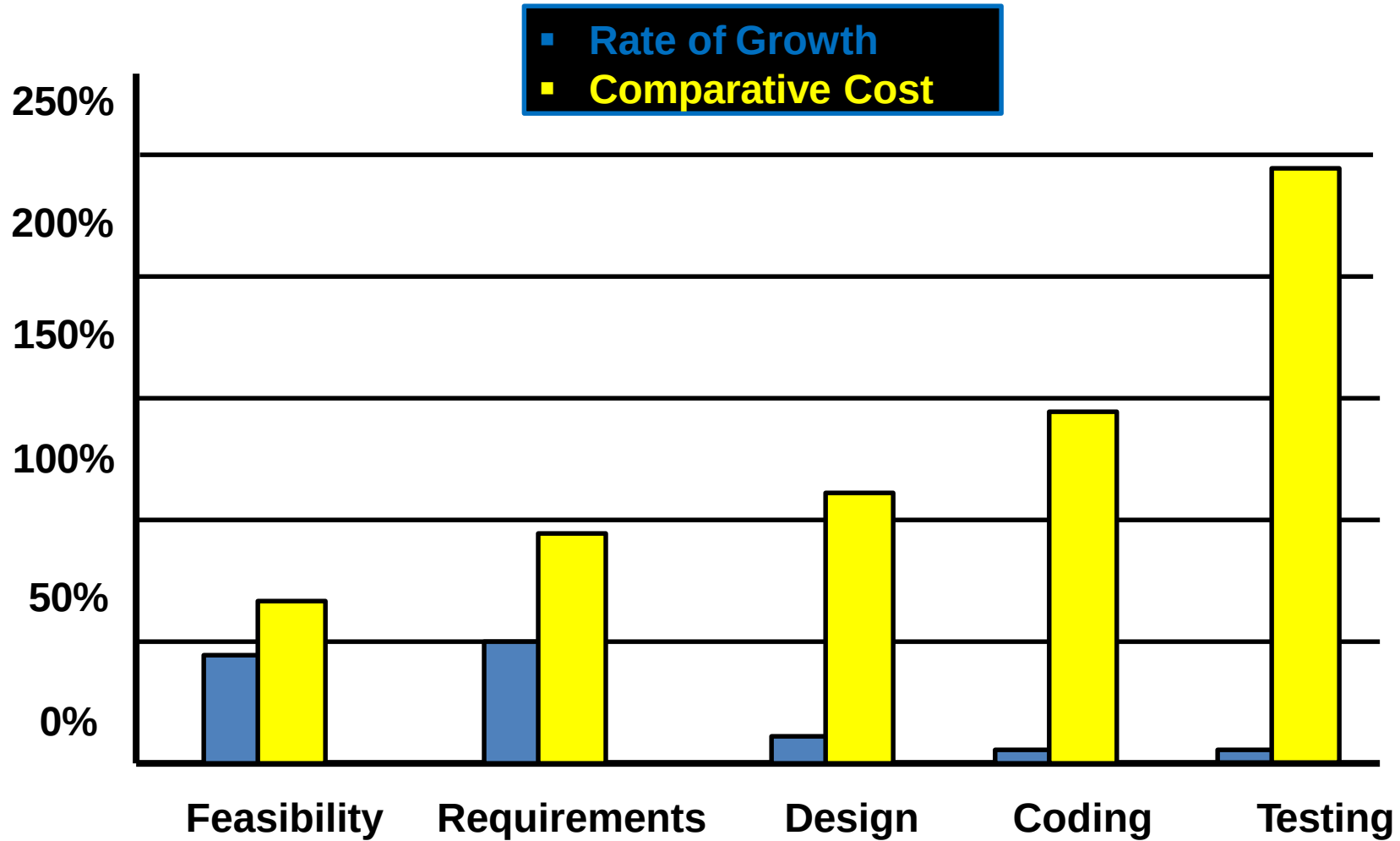
Design and Implementation Constraints

- They are **development guidelines** within which the designer must work, which can seriously **limit design** and **implementation** options
- Can also have **impact** on **human resources**

Relationship of Several components of Software Requirements



Requirement Origin v/s Costs



Some Risks From Inadequate Requirement Process

- Insufficient user involvement leads to unacceptable products
- Creeping user requirements contribute to overruns and degrade product quality
- Ambiguous requirements lead to ill-spent time and rework
- Gold-plating by developers and users adds unnecessary features

Some Risks From Inadequate Requirement Process

- Minimal specifications lead to missing **key requirements**
- **Overlooking** the needs of certain stake holders leads to dissatisfied customers
- **Incompletely** defined requirements make accurate **project planning and tracking impossible**

Ambiguous Requirements

- The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized.
- Allows user to cancel sales orders.
- The system provides searching of data quickly.
- “The system is user friendly.”

Ambiguous Requirements

- “Software produces a print normally in ten seconds.”
- The staff shall be able to use all the system functionalities. The average number of errors made by users shall not exceed two per hour of system use.

Ambiguous Requirements

- “The operator identity consists of the operator name and password; the password consists of six digits. It should be displayed on the security screen and deposited in the login file when an operator logs into the system.”

Characteristics of Good Requirements

- Numbered
- Inspected
- Unambiguous
- Testable
- Complete
- Consistent
- Understandable
- Traceable
- Feasible
- Modifiable

Task - 1

Write down five functional requirements and five non-functional requirements of a mobile telephone.



Task - 2

Write down five functional requirements and five non-functional requirements of an **online examination system**.



Quality Factors

- Portability
- Reusability
- Interoperability
- Survivability
- Safety
- Manageability
- Supportability
- Replaceability
- Functionality
- Correctness
- Reliability
- Efficiency
- Usability
- Integrity
- Maintainability
- Flexibility
- Testability
- Security

The Problems with our Requirements Practices

- We have trouble understanding the requirements that we do acquire from the customer
- We often record requirements in a disorganized manner
- We spend far too little time verifying what we do record
- We allow change to control us, rather than establishing mechanisms to control change
- Most importantly, we fail to establish a solid foundation for the system or software that the user wants built

(more on next slide)

Requirements Engineering Tasks

- Seven distinct tasks
 - Inception
 - Elicitation
 - Elaboration
 - Negotiation
 - Specification
 - Validation
 - Requirements Management
- Some of these tasks **may occur in parallel** and all are adapted to the needs of the project
- All strive to define what the customer wants
- All serve to establish a solid foundation for the design and construction of the software

Inception

Elicitation

Elaboration

Negotiation

Specification

Validation

Requirements
Management

Inception Task

- During **inception**, the requirements engineer asks a set of questions to establish...
 - A basic understanding of the problem
 - The people who want a solution
 - The nature of the solution that is desired
 - The effectiveness of preliminary communication and collaboration between the customer and the developer
- Through these questions, the requirements engineer needs to...
 - Identify the stakeholders
 - Recognize multiple viewpoints
 - Work toward collaboration
 - Break the ice and initiate the communication



The First Set of Questions

These questions focus on the customer, other stakeholders, the overall goals, and the benefits

- Who is behind the request for this work?
- Who will use the solution?
- What will be the **economic benefit** of a successful solution?
- Is there another source for the solution that you need?

The Next Set of Questions

These questions enable the requirements engineer to gain a better understanding of the problem and allow the customer to voice his or her perceptions about a solution

- How would you characterize "good" output that would be generated by a successful solution?
- What problem(s) will this solution address?
- Can you show me (or describe) the business environment in which the solution will be used?
- Will special performance issues or constraints affect the way the solution is approached?

The Final Set of Questions

These questions focus on the effectiveness of the communication activity itself

- Are you the right person to answer these questions? Are your answers "official"?
- Are my questions relevant to the problem that you have?
- Am I asking too many questions?
- Can anyone else provide additional information?
- Should I be asking you anything else?

```
graph TD; A[Inception] --> B[Elicitation]; B --> C[Elaboration]; C --> D[Negotiation]; D --> E[Specification]; E --> F[Validation]; F --> G[Requirements Management];
```

Inception

Elicitation

Elaboration

Negotiation

Specification

Validation

Requirements
Management

Elicitation Task

- Eliciting requirements is difficult because of
 - **Problems of scope** in identifying the boundaries of the system or specifying too much technical detail rather than overall system objectives
 - **Problems of understanding** what is wanted, what the problem domain is, and what the computing environment can handle (Information that is believed to be "obvious" is often omitted)
 - **Problems of volatility** because the requirements change over time
- **Elicitation may be accomplished through two activities**
 - **Collaborative requirements gathering**
 - **Quality function deployment**

Basic Guidelines of Collaborative Requirements Gathering

- **Meetings** are conducted and attended by both **software engineers, customers, and other interested stakeholders**
- Rules for preparation and participation are established
- An **agenda** is suggested that is formal enough to cover all important points but informal enough to encourage the free flow of ideas
- A "**facilitator**" (customer, developer, or outsider) controls the meeting
- A "**definition mechanism**" is used such as work **sheets, flip charts, wall stickers, electronic bulletin board, chat room**, or some other virtual forum
- The **goal** is to identify the **problem, propose** elements of the solution, **negotiate** different approaches, and specify a preliminary set of solution requirements

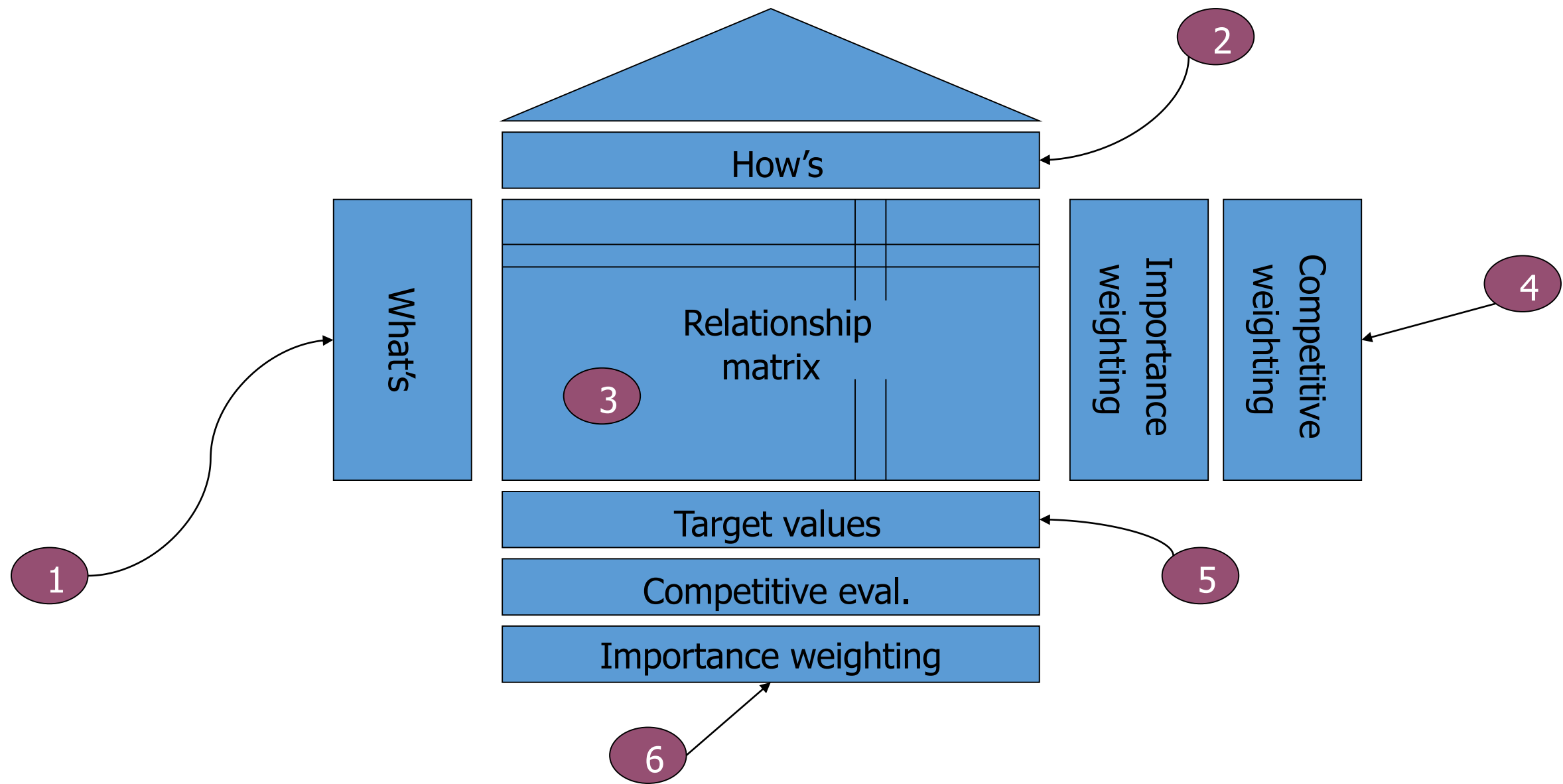
Quality Function Deployment

- This is a technique that translates the needs of the customer into technical requirements for software
- It emphasizes an understanding of what is valuable to the customer and then deploys these values throughout the engineering process through functions, information, and tasks
- It identifies three types of requirements
 - Normal requirements: These requirements are the objectives and goals stated for a product or system during meetings with the customer
 - Expected requirements: These requirements are implicit to the product or system and may be so fundamental that the customer does not explicitly state them
 - Exciting requirements: These requirements are for features that go beyond the customer's expectations and prove to be very satisfying when present

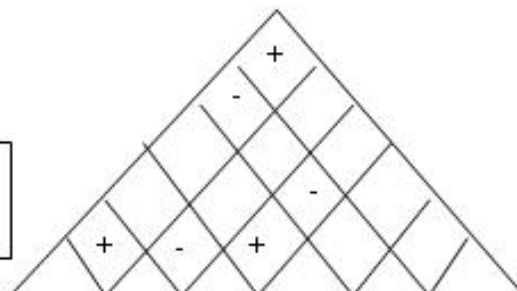
QFD process (1)

- The basic idea of QFD is to construct relationship matrices between customer needs, technical requirements, priorities and (if needed) competitor assessment.
- To achieve this the following process is prescribed:
 1. Identify stakeholder's attributes or requirements
 2. Identify technical features of the requirements
 3. Relate the requirements to the technical features
 4. Conduct an evaluation of competing products
 5. Evaluate technical features and specify a target value for each feature
 6. Prioritize technical features for development effort.

QFD process (2)



INTERACTIONS:
+ Positive Relationship
- Negative Relationship



	P R I O R I T Y	Product Design Requirements	Smaller Casing (length reduction)	Less Gears	Long Battery Life	Mammal Override	LED installed	Finger Grooves	Competitive Evaluation
Customer Requirements									
Safety	5		-			+	+	+	Δ ⊗ B
Durability	4		-						Δ B ⊗
Ease of Use	4		+		+	+	+	+	⊗B Δ
Cost +=less cost	3		+	+	+	-	-	-	Δ B ⊗
Weight +=less weight	3		+	+		-	-		⊗ B Δ
Battery Life	4			-	+		-		BA⊗
Technical Evaluation	5 1		Δ B ⊗	Δ ⊗B	Δ⊗B	B Δ⊗			RELATIONSHIPS: Δ Black & Decker ⊗ Handy Gourmet B One-Touch
Target Value			1.25in	2	100 cans	2 lb in	5,000 hrs	4 in.	
Technical Difficulty			3	2	3	5	4	3	
Importance Rating			2	3	4	4	4	3	

FIGURE 5.3

UML activity diagrams for eliciting requirements

